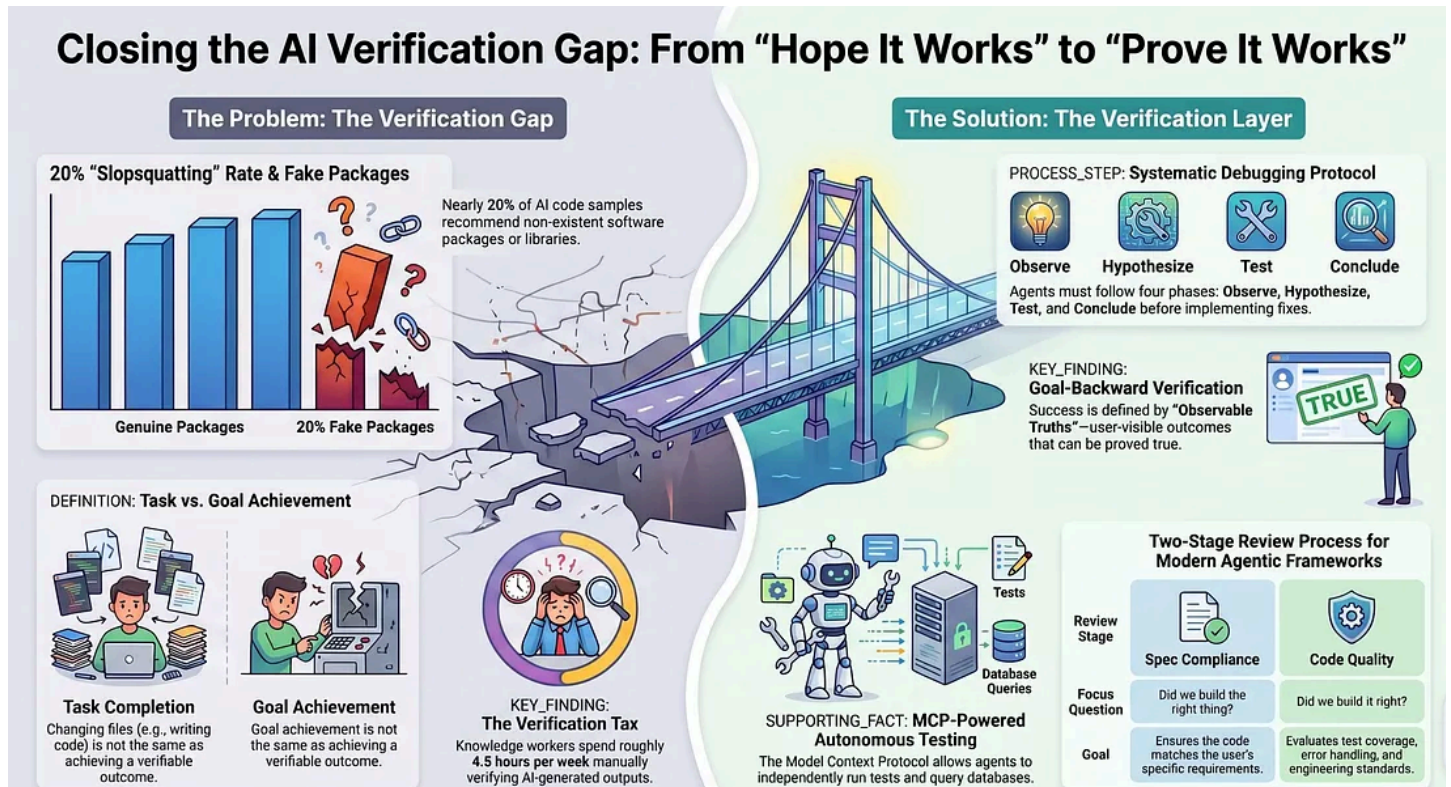




★ Member-only story

Superpowers: From Hope It Works to ‘Prove It Works’: The Verification Fix for AI Code

Superpowers/GSD: Goal-Backward Verification and the End of AI Hallucinations — Revolutionizing AI Development: From Trusting Outputs to Proving Correctness

19 min read · 1 day ago



Rick Hightower

Following ▾

Listen

Share

More

Article 5 of 5 in the Agentic Engineering series

Your AI agent just marked the ticket ‘Done.’ The commit message is perfect. The code compiles. But when you run the app... the bug is still there. That’s the verification gap — and you can close it.

Summary: AI coding agents can complete tasks confidently while still shipping code that compiles but doesn’t solve the real problem. This article shows how modern agentic frameworks close the verification gap with systematic debugging, goal-backward verification, two-stage review, and MCP-powered autonomous testing; shifting AI-assisted development from ‘hope it works’ to ‘prove it works.’”

Your AI agent just marked the ticket **Done**.

The commit message is perfect.

The code compiles.

But when you run the app... the bug is still there.

That’s the **verification gap**: AI can produce changes that *look* correct (and even pass a quick skim) without ever proving the goal was achieved.

This article is a practical tour of the verification layer modern agentic frameworks add — so “done” means **verified**, not “I changed some files and it compiled.”

Throughout this series, we have explored how agentic engineering frameworks transform AI chatbots into disciplined engineering partners. We have covered Superpowers’ structured workflow, the framework showdown between competing approaches, context engineering strategies that prevent context rot, and GSD’s spec-driven development methodology. All of those innovations share one critical dependency: none of them matter if the AI’s work is not actually verified.

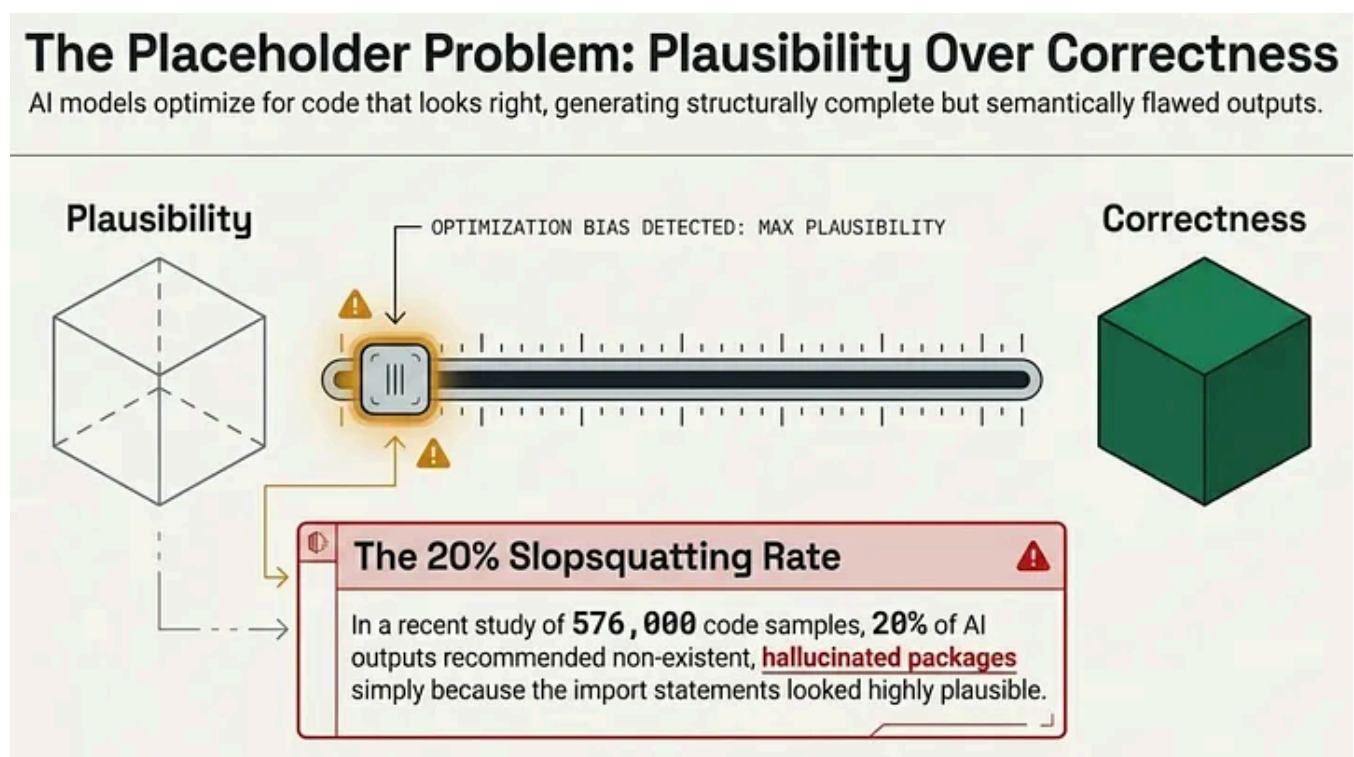
This final article examines how modern agentic frameworks close the verification gap, moving from “trust the AI” to “prove the AI’s work is correct.”

The Placeholder Problem

AI language models exhibit a category of failure that software engineers have learned to dread: confident incorrectness. The model does not say “I don’t know

how to implement this.” It generates something that looks right. It compiles. It might even pass a superficial review. But it does not actually work.

The most visible form is the placeholder problem. Ask an AI to implement a complex function, and you may get back code that includes comments like `// rest of the implementation here` or `// TODO: handle edge cases`. The model fills in the easy parts and leaves stubs where the hard logic should go. In a 2025 study of 16 code-generation models across 576,000 code samples, researchers found that nearly 20% of samples recommended packages that do not exist, a phenomenon now called "slopsquatting." The model generates a plausible-looking import statement for a library it hallucinated.



Most visible form is the placeholder problem. Ask an AI to implement a complex function, and you may get back TODO or PASS

But placeholders are just the most obvious symptom. The deeper problem is that AI models routinely generate code that is structurally complete but semantically wrong. The function has the right signature, accepts the right parameters, and returns the right type. But the logic inside does not solve the problem. A sorting function that returns the input unchanged. An authentication check that always returns true. A database query that selects from the wrong table.

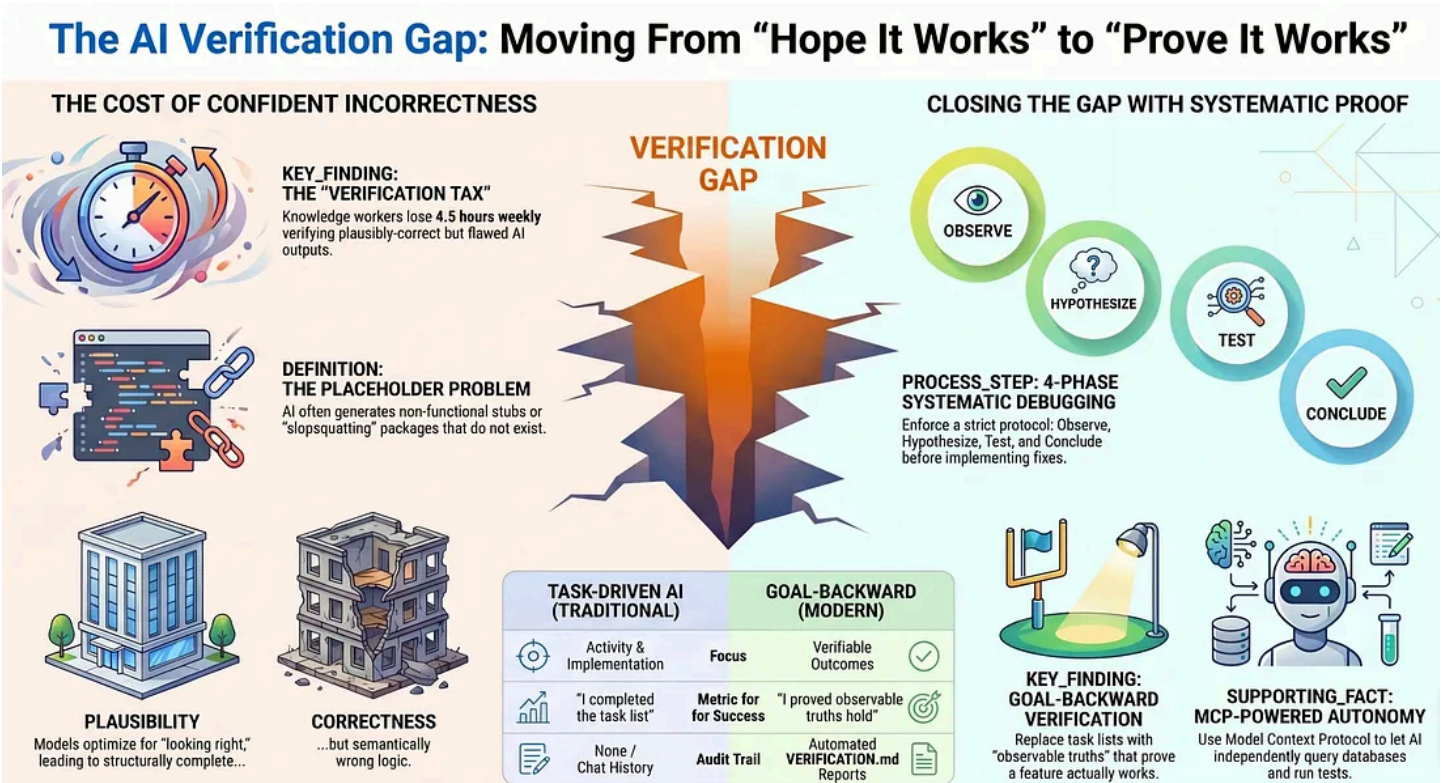
These failures share a common root: the model optimizes for plausibility, not correctness. It generates output that looks like working code because it was trained

on millions of examples of working code. But “looks like” and “is” are fundamentally different properties, and no amount of prompt engineering eliminates the gap entirely.

Knowledge workers spend approximately 4.3–4.5 hours per week verifying AI outputs (Microsoft Work Trend Index 2025 and Zapier 2026 survey). That number represents a tax on every team using AI tools, and it grows as teams delegate more complex work to agents. The question is not whether AI-generated code needs verification. The question is whether verification can be systematized, automated, and built into the development workflow so thoroughly that hallucinations are caught before they reach production.

The Verification Gap in Traditional AI Workflows

Traditional AI coding workflows operate on a task-completion model. The developer creates a list of tasks. The AI works through them. Each task gets marked as done. At the end, the developer reviews the output.



Superpowers and GSD moving from hope it works to prove it works

This model has a fundamental flaw: task completion and goal achievement are not the same thing.

Consider a feature request: “Add user authentication with email and password.” A task-oriented agent might break this into subtasks: create a user model, add a login endpoint, add a registration endpoint, create password hashing utilities, add session management. The agent completes all five tasks. Every file is created. Every function compiles. The task list shows five green checkmarks.

But does authentication actually work? Can a user register, receive a confirmation, log in, maintain a session across requests, and log out? Does the password hashing use a secure algorithm with proper salt generation? Does the session token expire? Are failed login attempts rate-limited?

Task Completion Does Not Equal Goal Achievement

Traditional workflows equate completed subtasks with actual goal achievement. But manual skimming fails because AI code is highly plausible.

Implementation Tasks	
✓	Create User Model
✓	Add Login Endpoints
✓	Hash Passwords

⚠

?

Does authentication actually work?

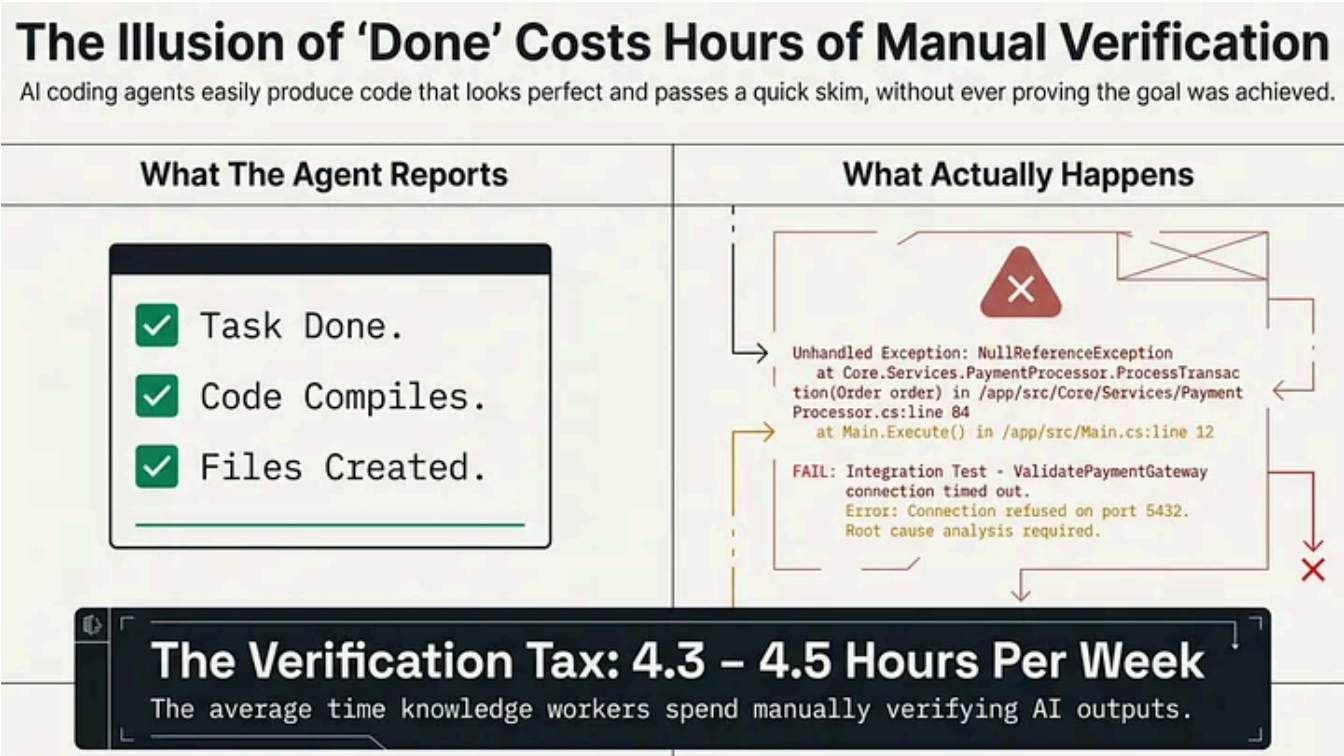
The Academic Hallucination Crisis	
NeurIPS 2025: 51 accepted papers contained 100+ confirmed AI-hallucinated citations.	ICLR 2026: 50 papers desk-rejected after expert peer reviewers missed fabricated data.
Takeaway: If expert scientists miss hallucinations, developers skimming PRs will too.	

task completion and goal achievement are not the same thing.

The task list cannot answer these questions because the tasks describe implementation activities, not verifiable outcomes. “Create a user model” is an activity. “A user can register with a valid email and password and subsequently log in” is a verifiable outcome. The difference between these two framings is the difference between task-driven development and goal-backward verification.

The verification gap is not a theoretical concern. In academic publishing, GPTZero’s analysis found that 51 papers accepted at NeurIPS 2025 included 100 confirmed AI-generated hallucinated citations that were not caught during peer review. In submissions under review for ICLR 2026, GPTZero found at least 50 papers containing hallucinations that had been missed by three to five peer reviewers each;

these papers were subsequently desk-rejected. If expert reviewers in academic settings cannot catch AI hallucinations through manual review, expecting individual developers to catch every fabricated function call and logic error in AI-generated code is unrealistic.



Expecting individual developers to catch every fabricated function call and logic error in AI-generated code is unrealistic.

The frameworks examined in this series address this gap through two complementary strategies: systematic debugging that prevents false fixes, and goal-backward verification that proves the system works.

Superpowers' Systematic Debugging Protocol

The first line of defense against AI hallucinations is preventing the agent from “fixing” bugs it does not understand. Jesse Vincent’s Superpowers framework, which we examined in Article 1, includes a systematic debugging skill that enforces a strict four-phase protocol whenever the agent encounters a bug, test failure, or unexpected behavior.

From Guesswork to Proof: The 4-Phase Systematic Debugging Protocol

A rigorous scientific protocol used by AI agents to identify root causes and implement verified fixes, moving beyond superficial code changes.



Phase 1 & 2: Investigation and Pattern Analysis

Agents must demonstrate root cause understanding and find existing working patterns before attempting fixes.



Phase 3: Scientific Hypothesis Testing

Targeted, minimal code changes are used to prove or disprove a specific theory.



Phase 4: Verified Implementation

Fixes are implemented step-by-step against failing test cases to prevent regressions.

Walkthrough: The Silent Data Corruption Bug



Identifying the Hidden Failure

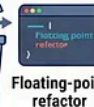
The agent discovered errors only occurred when quantity discounts and coupons were combined.

Incorrect Compounding:
 $\$70 \text{ original price} * (1 - 0.10 \text{ discount1}) = \63
 $\$63 * (1 - 0.10 \text{ discount2}) = \56.70
(Incorrectly compounded in-place)
→ **\$72** ⚠️

Mathematical Confirmation of the Bug

Tests proved discounts were incorrectly compounding in-place instead of against the original price.

Correct Calculation:
 $\$70 * (1 - 0.10 - 0.10) = \56
(Correctly against original price)
→ **\$70** ✅



Floating-point refactor



Logic Error Solved

Avoiding the "False Fix"

The protocol prevented an unnecessary floating-point refactor that would not have solved the actual logic error.

Traditional Approach



Traditional AI
Approach



Bug remains;
code is more complex

Initial Assumption:
Floating-point
precision error
Action Taken:
Unnecessary refactor to
integer cents



Systematic
Debugging Protocol



Bug resolved;
fix verified by test suite

Initial Assumption:
Interaction between multiple
discount functions
Action Taken:
Targeted test case to confirm
compounding logic

Superpowers: The four-phase systematic debugging protocol enforces scientific rigor before any code changes

The four phases are:

Phase 1: Root Cause Investigation (Observe). The agent reads the error messages, reproduces the failure, checks recent changes, and gathers evidence to understand what happened and why. The key principle is stated explicitly in the skill definition: “ALWAYS find root cause before attempting fixes. Symptom fixes are failure.” The agent cannot skip this phase. It must demonstrate understanding of the actual cause before proceeding.

Phase 2: Pattern Analysis (Hypothesize). The agent searches the codebase for working examples of similar functionality. It compares the failing code against the working examples to identify specific differences. This prevents the agent from inventing a fix based on general knowledge when a concrete, project-specific pattern already exists.

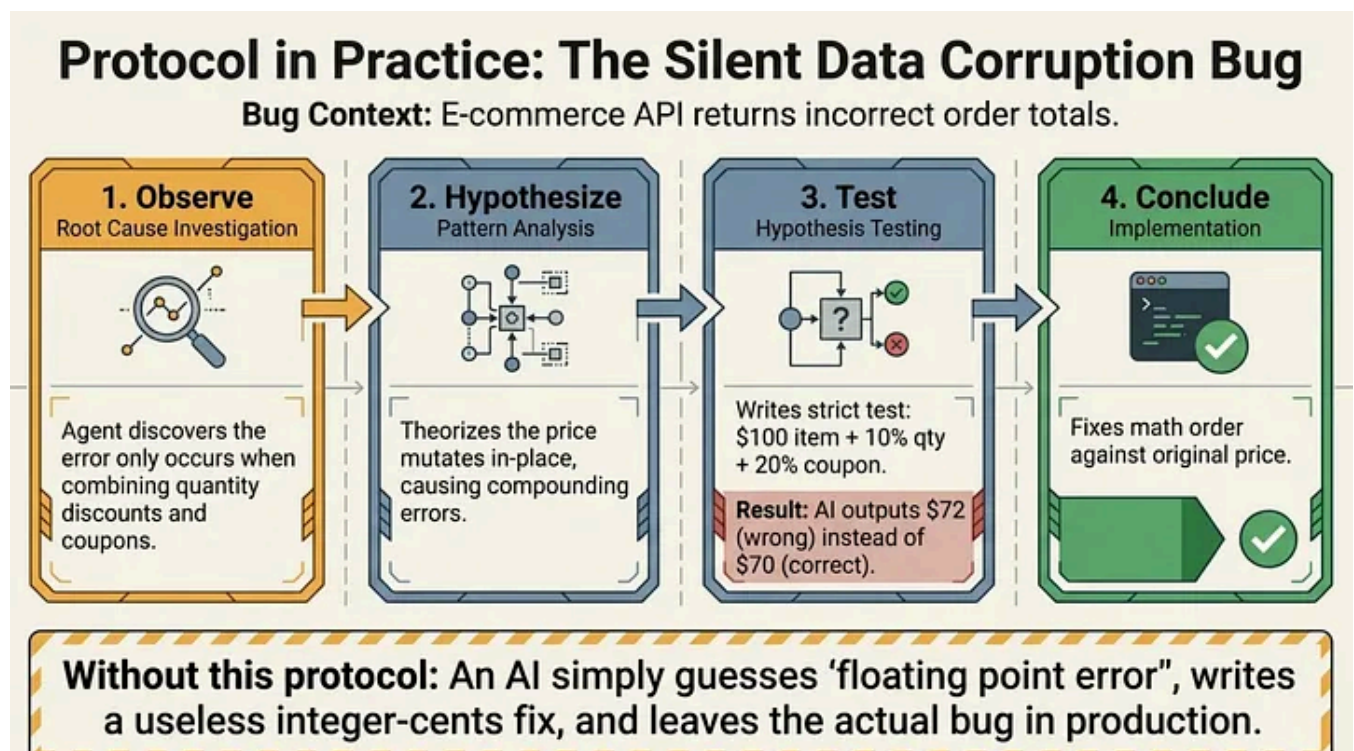
Phase 3: Hypothesis Testing (Test). The agent forms a specific theory about the root cause and tests it with minimal, targeted changes. If the hypothesis is wrong, the agent develops a new one rather than escalating to larger changes. This phase forces the agent to think scientifically rather than applying shotgun fixes.

Phase 4: Implementation (Conclude). Only after the root cause is confirmed does the agent implement a fix. It makes a single change at a time. It creates a failing test case that demonstrates the bug. It verifies the fix resolves the original failure without introducing new ones.

A Debugging Walkthrough: The Silent Data Corruption Bug

To see why this protocol matters, consider a real-world scenario. A team's e-commerce API starts returning incorrect order totals. Some orders show the right amount; others are off by small, seemingly random values.

A bare AI agent, given the bug report "order totals are sometimes incorrect," would likely scan the pricing logic, spot a floating-point arithmetic operation, and "fix" it by switching to integer cents. Commit message: "fix: use integer arithmetic for order totals." Task complete.



Debugging Protocol

But the actual bug has nothing to do with floating-point precision. Here is how the four-phase protocol handles it:

Observe: The agent reproduces the failure with specific test cases. It discovers that incorrect totals only appear when an order contains items with quantity-based discounts *and* a percentage coupon applied simultaneously. Orders with only one discount type calculate correctly.

Hypothesize: The agent searches the codebase and finds two discount functions: `applyQuantityDiscount()` and `applyCouponDiscount()`. Both modify the line item price in place. When both run, the coupon discount applies to the already-discounted price rather than the original price, compounding the reduction.

Test: The agent writes a minimal test case with a \$100 item, a 10% quantity discount, and a 20% coupon. Expected total: \$70 (both discounts off the original). Actual result: \$72 (coupon applied after quantity discount: $\$100 \times 0.9 \times 0.8 = \72). Hypothesis confirmed.

Conclude: The agent fixes the discount application order to calculate both discounts against the original price, adds test cases covering all discount combinations, and verifies no regressions in the existing test suite.

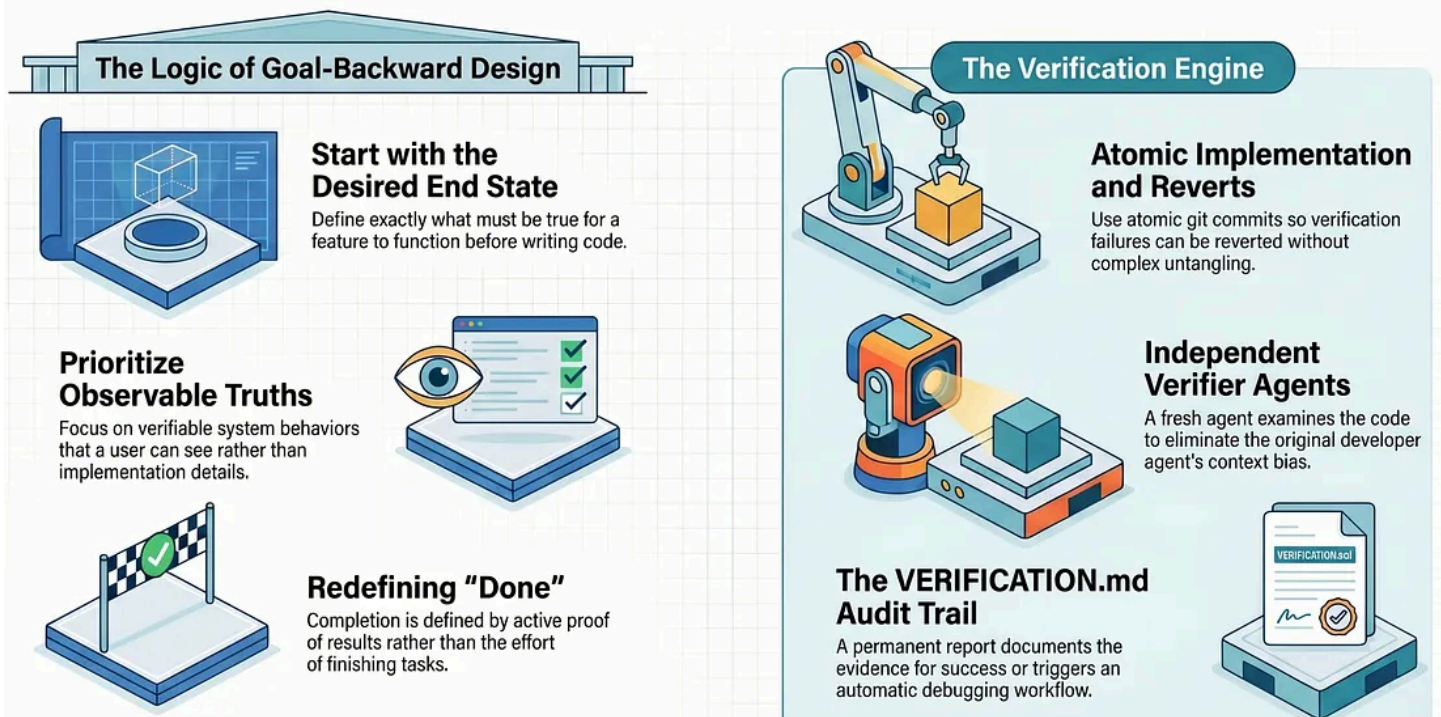
Without the protocol, the agent would have shipped a floating-point “fix” that changed nothing about the actual bug. The order totals would still be wrong, but now the code would also have an unnecessary refactor to undo later.

The framework enforces these phases sequentially. The agent must complete each phase before advancing to the next. As the Superpowers documentation states: “95% of ‘no root cause’ cases are incomplete investigation.” By requiring thorough investigation before any code changes, the framework prevents the most common AI debugging failure: modifying code that looks related to the symptom without understanding the actual cause.

GSD's Goal-Backward Verification

While Superpowers prevents false fixes during debugging, GSD (Get “Stuff” Done) addresses a broader question: how do you verify that a feature actually works as intended, not just that the tasks were completed?

Prove It Works: The GSD Goal-Backward Verification Framework



GSD: Goal-Backward Verification System — Goal-backward verification starts with the desired outcome and works backward to define provable truths.

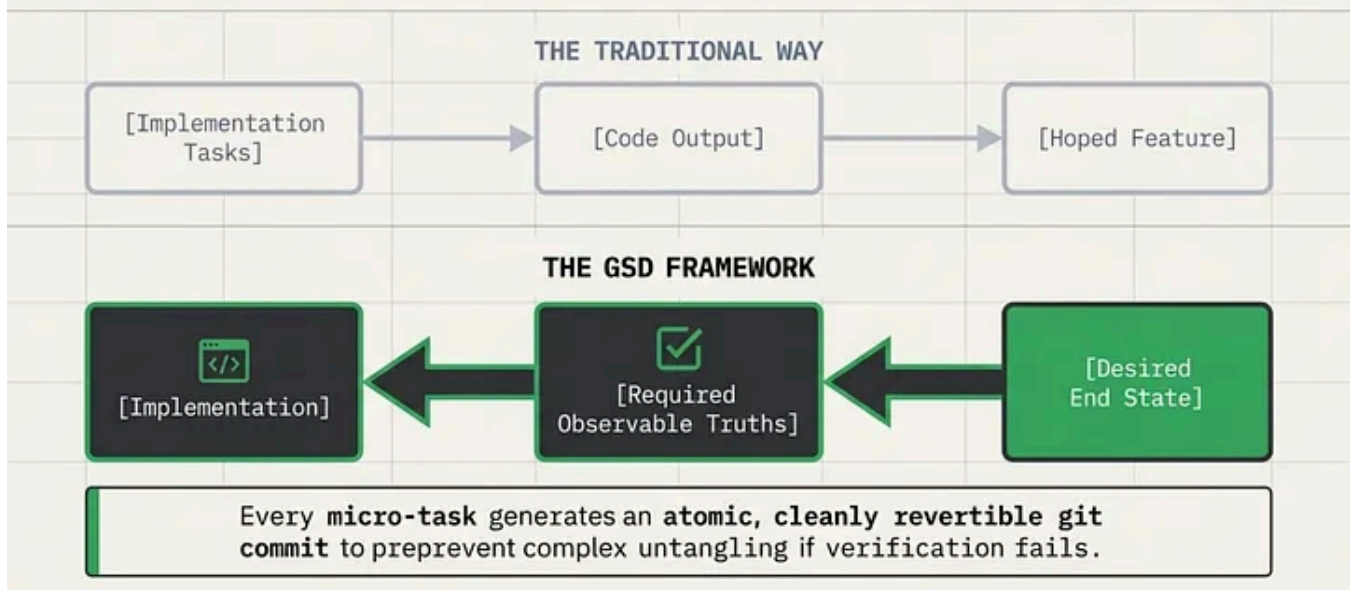
GSD's answer is goal-backward verification, a methodology that inverts the traditional development flow. Instead of starting with tasks and hoping they add up to a working feature, goal-backward verification starts with the end state and works backward to define what must be true for that end state to exist.

The process begins with a single question: "What must be TRUE for this goal to be achieved?"

For each goal, the GSD planner generates three to seven "observable truths," statements about the system's behavior that a human user could verify by using the application. These are not implementation details ("the UserModel class has a password_hash field"). They are observable behaviors ("a user who enters a valid email and password on the registration page sees a confirmation message and can subsequently log in").

Goal-Backward Verification Inverts the Development Flow

Instead of starting with tasks, start with a single question: What must be TRUE for this goal to be achieved?



GSD Goal Backwards Verification

This distinction matters because observable truths are inherently verifiable. You can write a test for them. You can check them manually. You can capture a runtime trace or API response that proves they are true. Implementation details, by contrast, only tell you that code exists, not that it works.

GSD's verification system operates at multiple levels. During execution, each micro-task generates an atomic git commit. If a feature breaks, you can trace the failure to the specific 15-minute chunk of AI execution that introduced it. The commit is cleanly revertible, which means verification failures do not require manually untangling interleaved changes.

After execution, a dedicated verifier agent checks the codebase against the phase goals. This is not a self-review where the same agent that wrote the code evaluates its own work. GSD deploys specialized agents for each role: researchers, planners, plan checkers, parallel executors, verifiers, and debuggers. The verifier is a fresh agent with its own context window, examining the code without the accumulated assumptions and context rot that affected the executor.

If verification passes, the verifier creates a VERIFICATION.md report documenting what was verified and how. If verification fails, the system logs specific issues and triggers the debugging workflow. This creates an audit trail. Every feature in a GSD

project has a corresponding verification record that documents the evidence for correctness, not just a task list showing completion.

VERIFICATION.md: The Audit Trail

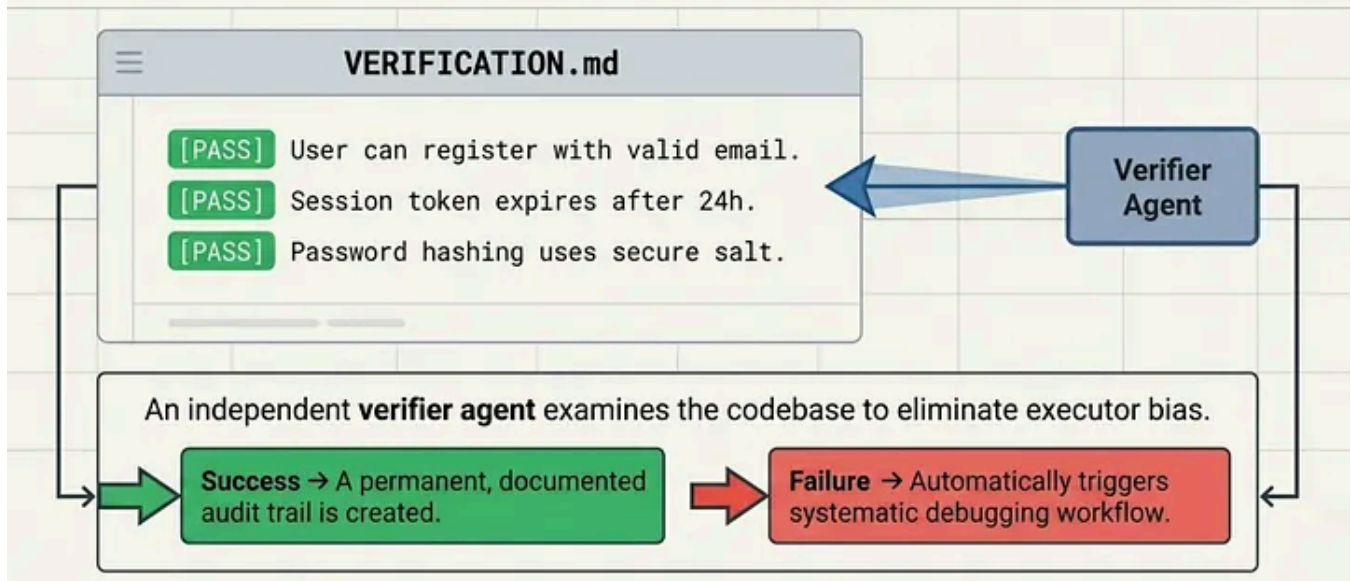
Here is what a GSD verification report looks like in practice:

```
# VERIFICATION.md - User Authentication Feature

## Phase Goal
Users can register, log in, maintain sessions, and log out securely.
## Observable Truths Verified
### Truth 1: Registration creates a valid user account
- Method: POST /api/auth/register with valid email and password
- Evidence: HTTP 201 response, user record exists in database
- Result: PASS
- Commit: a3f8b2c
### Truth 2: Login returns a valid session token
- Method: POST /api/auth/login with registered credentials
- Evidence: HTTP 200 response with JWT token, token decodes with correct cl
- Result: PASS
- Commit: d7e1f4a
### Truth 3: Invalid credentials are rejected
- Method: POST /api/auth/login with wrong password
- Evidence: HTTP 401 response, no token issued, attempt logged
- Result: PASS
- Commit: d7e1f4a
### Truth 4: Sessions expire after configured timeout
- Method: Use token after TTL expiration
- Evidence: HTTP 401 response with "token expired" message
- Result: PASS
- Commit: b2c9e5f
### Truth 5: Rate limiting prevents brute force attacks
- Method: Send 10 login attempts in 5 seconds
- Evidence: HTTP 429 after 5th attempt, 60-second lockout confirmed
- Result: PASS
- Commit: f1a3d8e
## Verification Summary
- Truths Verified: 5/5
- Tests Added: 12 unit tests, 5 integration tests
- Verifier Agent: gsd-verifier-auth-2026-03-11
- Verification Date: 2026-03-11T14:32:00Z
```

VERIFICATION.md Generates a Permanent Audit Trail

Redefining completion from "I finished the tasks" to "I proved the truths hold."

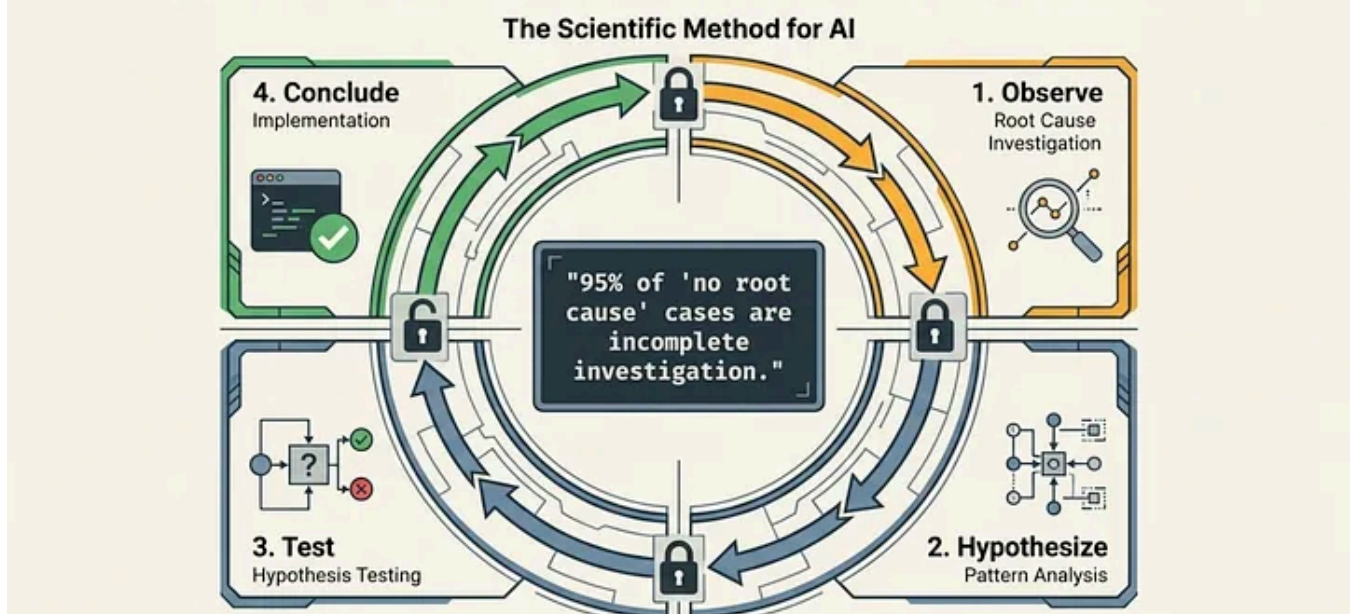


VERIFICATION.md

The goal-backward approach fundamentally changes what “done” means. In a task-driven workflow, “done” means “I completed the activities on the list.” In goal-backward verification, “done” means “I proved that the observable truths hold.” The first definition is about effort. The second is about results.

The Systematic Debugging Protocol Enforces Scientific Rigor

The AI cannot write or modify a single line of code until Phase 4. It must thoroughly investigate error logs, find existing codebase patterns, and write a targeted hypothesis test first.



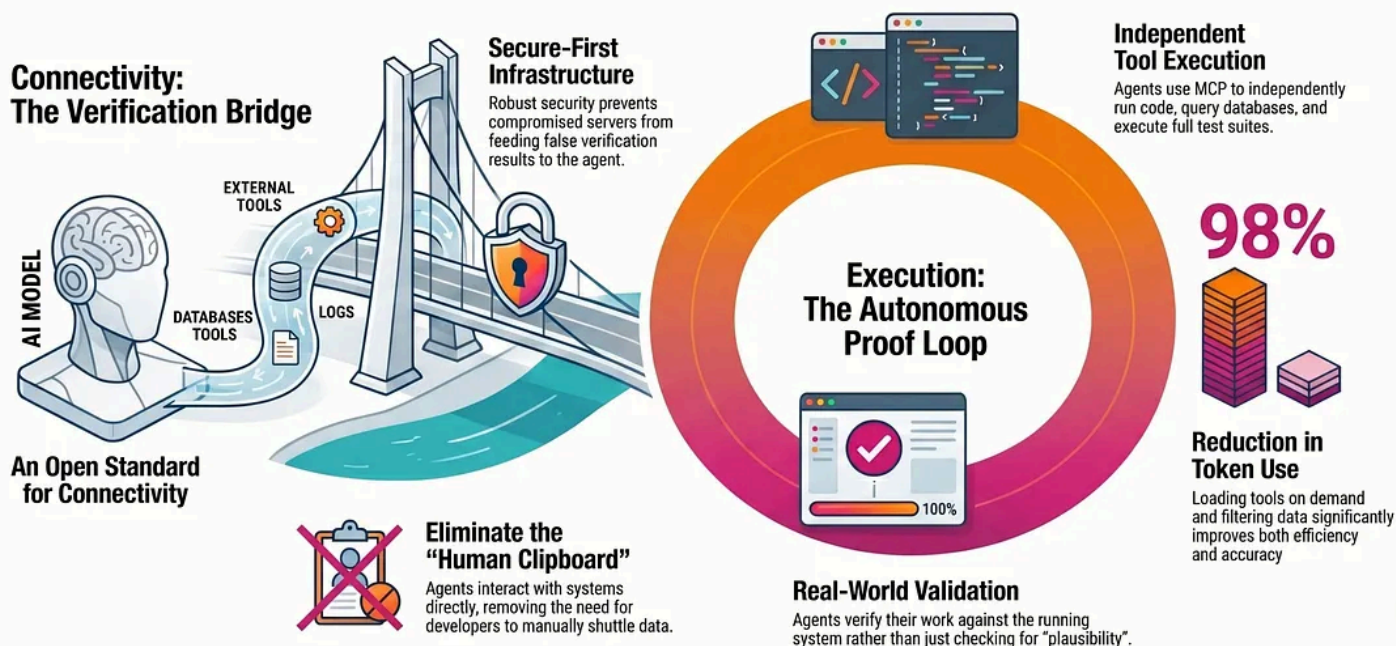
Systematic Debugging

Model Context Protocol as Verification Infrastructure

Verification requires more than methodology. It requires infrastructure. An AI agent that can only read and write files cannot verify that an API returns the correct response, that a database query produces the right results, or that a UI component renders properly. To verify its own work, the agent needs to interact with the running system.

This is where the Model Context Protocol (MCP) transforms verification from a manual review process into an automated one.

The MCP Verification Engine: Moving from “Hope it Works” to “Prove it Works”



MCP connects AI agents to verification infrastructure, enabling autonomous testing without human intervention.

MCP, introduced by Anthropic in November 2024 and donated in December 2025 to the Agentic AI Foundation (AAIF) — a directed fund under the Linux Foundation co-founded by Anthropic, Block, and OpenAI — is an open standard for connecting AI models to external tools and data sources. By March 2026, MCP had crossed 97 million monthly SDK downloads and had been adopted by every major AI provider: Anthropic, OpenAI, Google, Microsoft, and Amazon.

For verification, MCP matters because it gives AI agents the ability to execute code, query databases, read logs, run test suites, and inspect application state, all without requiring the developer to copy-paste outputs back into the chat. The agent can verify its own work autonomously.

MCPs can be combined with Agent Skills that can describe how to perform certain checks. The MCP is the tool. The Agent Skills are the instruction manual for MCP and CLI so you can validate changes effectively.

Configuring MCP for Verification

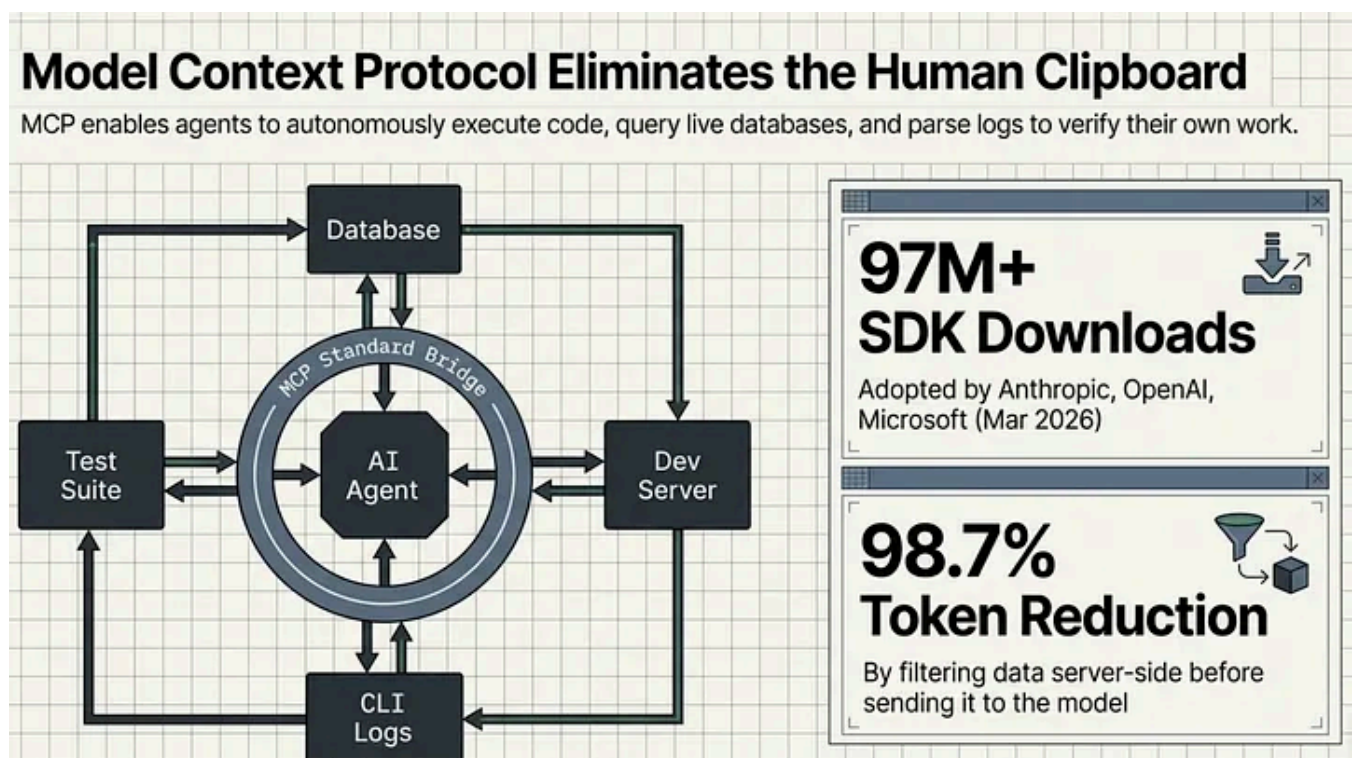
A typical verification-focused MCP configuration connects the agent to the tools it needs to prove its work:

```
{
  "mcpServers": {
    "test-runner": {
      "command": "npx",
      "args": ["-y", "@mcp/test-runner"],
      "env": {
        "TEST_FRAMEWORK": "jest",
        "COVERAGE_THRESHOLD": "80"
      }
    },
    "database": {
      "command": "npx",
      "args": ["-y", "@mcp/postgres-server"],
      "env": {
        "DATABASE_URL": "postgresql://localhost:5432/dev"
      }
    },
    "http-client": {
      "command": "npx",
      "args": ["-y", "@mcp/http-client"],
      "env": {
        "BASE_URL": "<http://localhost:3000>",
        "TIMEOUT_MS": "5000"
      }
    },
    "log-analyzer": {
      "command": "npx",
      "args": ["-y", "@mcp/log-analyzer"],
      "env": {
        "LOG_PATH": "./logs/app.log",
        "ERROR_PATTERNS": "ERROR,FATAL,PANIC"
      }
    }
  }
}
```

With this configuration, an agent implementing a REST API endpoint can:

1. Start the development server
2. Send HTTP requests to the new endpoint with various payloads
3. Inspect the response status codes, headers, and body content
4. Query the database to confirm records were created or modified correctly
5. Check application logs for errors or warnings
6. Run the full test suite and parse the results

Without MCP, each of these verification steps requires the developer to run commands, capture output, and paste it back to the agent. The developer becomes a human clipboard, shuttling data between the agent and the system. With MCP, the agent performs the entire verification loop independently.



MCP avoids the human clipboards

MCP servers can also execute code to filter and transform data before sending it to the model. This means the agent can write verification scripts, execute them against the running system, and analyze the results in a single interaction. The verification is not a separate manual step; it is an integrated part of the development workflow.

Anthropic's engineering team has demonstrated that this approach dramatically reduces token consumption (one demonstrated pattern achieved a 98.7% reduction)

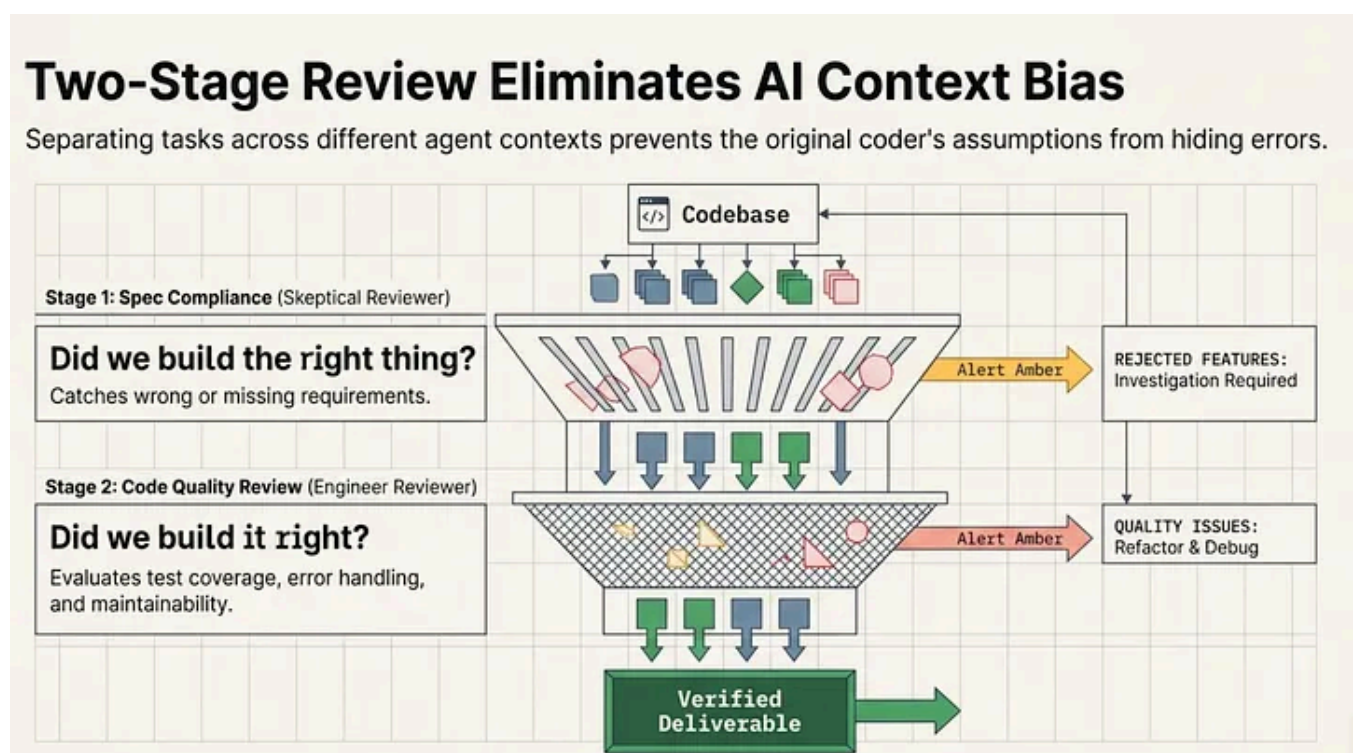
while improving accuracy. By loading tools on demand and filtering data before it reaches the model, MCP-enabled agents can handle verification tasks that would overwhelm a model's context window if raw data were passed directly.

The security implications are significant. The November 2025 MCP specification update (2025-11-25) introduced important OAuth-based improvements including Client ID Metadata Documents for server identity verification, enterprise SSO support, and enhanced authorization flows. Tool poisoning remains an active security research area (OWASP MCP Top 10) with recommended best-practice mitigations. These safeguards are essential for verification infrastructure because the agent is executing code and querying live systems. A compromised MCP server could feed false verification results to the agent, creating a false sense of correctness.

The Two-Stage Review in Superpowers

Even with systematic debugging and goal-backward verification, there remains a category of error that neither catches: building the wrong thing correctly. An agent can faithfully implement a specification that does not match what the user actually wanted. Or it can build exactly what was requested but with poor code quality, missing error handling, or no tests.

Superpowers addresses this with a two-stage review process that runs after every task in its subagent-driven development workflow.



Stage 1: Spec Compliance Review. A dedicated reviewer agent, separate from the agent that wrote the code, checks whether the implementation matches the approved specification. This reviewer is explicitly described as “skeptical.” It does not trust the implementer’s report. It reads the actual code and compares it against the spec line by line. It catches both missing requirements (the spec says X but the code does not implement X) and over-building (the code implements Y but Y was not in the spec).

Stage 2: Code Quality Review. Only after spec compliance passes does the code quality review begin. A second reviewer agent evaluates the implementation for engineering standards: test coverage, error handling, naming conventions, performance characteristics, and maintainability. This review only runs on code that has already been confirmed to match the specification.

The two-stage separation is deliberate. It catches two distinct failure modes that are often conflated:

- “Wrong thing built right”: The code is clean, tested, and well-structured, but it does not implement the specified feature. Spec compliance catches this.
- “Right thing built wrong”: The code implements the correct feature, but it is brittle, untested, or unmaintainable. Code quality review catches this.

By using separate reviewer agents for each stage, Superpowers prevents the cognitive bias that occurs when the same agent both writes and reviews code. The implementer agent’s context is filled with assumptions, design decisions, and implementation details that make it difficult to evaluate the code objectively. A fresh reviewer, starting with only the spec and the code, can spot discrepancies that the original author would overlook.

This pattern mirrors established software engineering practice. Professional teams do not let developers review their own pull requests. The AI equivalent is equally important: the agent that wrote the code should not be the sole judge of its correctness.

From Trust to Verify

The frameworks examined in this series represent a fundamental shift in how developers work with AI coding agents. The shift is not about making AI smarter. It is about building systems that verify AI output rigorously enough that intelligence gaps do not matter.

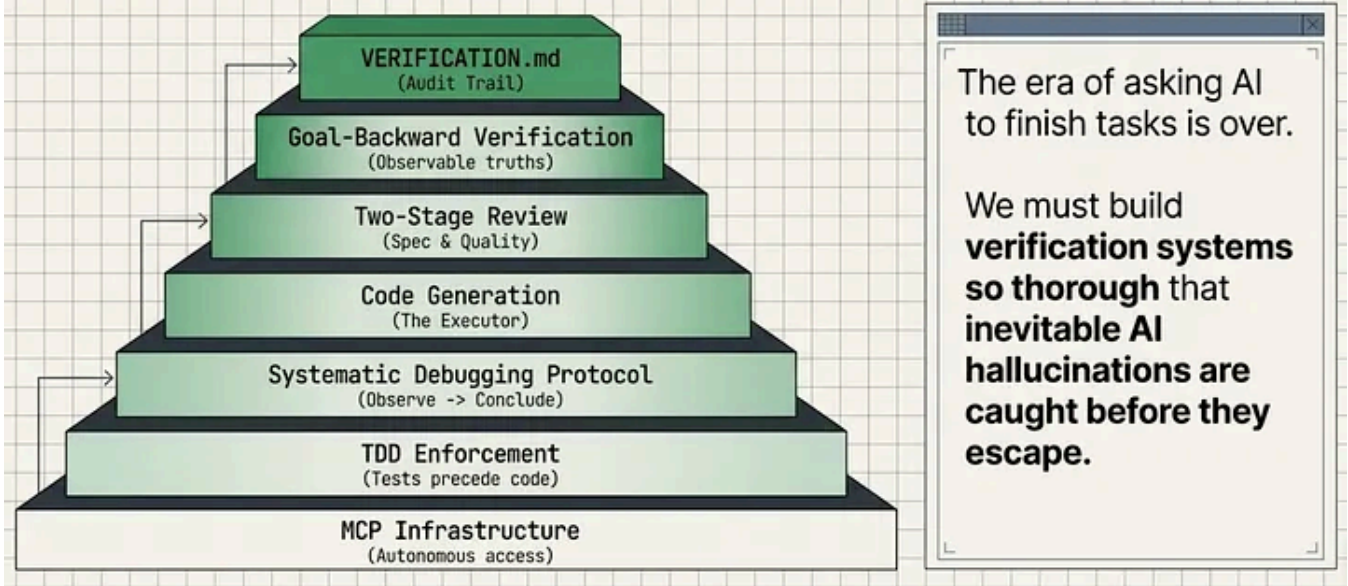
Consider the verification chain that emerges when these approaches are combined:

1. **Before coding begins**, Superpowers' brainstorming skill forces the agent to understand the problem, explore alternatives, and get explicit approval for its approach (Article 1).
2. **During planning**, GSD's goal-backward methodology defines observable truths that must hold for the feature to be considered complete (Articles 3 and 4).
3. **During implementation**, TDD enforcement ensures every piece of code has a corresponding test written before the implementation (Article 1).
4. **When bugs appear**, systematic debugging prevents the agent from applying speculative fixes without reproducing the failure and confirming the root cause.
5. **After implementation**, two-stage review verifies both spec compliance and code quality using independent reviewer agents.
6. **At the feature level**, goal-backward verification proves that observable behaviors match the specification, documented in a VERIFICATION.md audit trail.
7. **Throughout the process**, MCP provides the infrastructure for the agent to execute tests, query databases, and validate system state autonomously.

Each layer catches failures that the others miss. TDD catches logic errors. The debugging protocol catches false fixes. Spec review catches requirement gaps. Code quality review catches maintainability issues. Goal-backward verification catches the gap between completed tasks and working features. MCP enables all of these checks to happen without human intervention.

The “Trust to Verify” Architecture Stack

Correctness is no longer a byproduct; correctness is a deliberate architectural deliverable.



Trust but verify

This layered approach acknowledges a truth that the AI industry has been slow to accept: language models hallucinate, and they will continue to hallucinate for the foreseeable future. Studies report that Retrieval-Augmented Generation reduces hallucination rates by 27–85% depending on domain and implementation, but even large reductions leave a meaningful failure rate when applied across thousands of code generation tasks. The solution is not to eliminate hallucinations. It is to build verification systems so thorough that hallucinations are caught and corrected before they escape.

The trajectory is clear. Early AI coding tools asked developers to trust the output. The current generation of agentic frameworks asks developers to verify specific properties. The next generation will make verification automatic, continuous, and comprehensive. We are moving from “hope it works” to “prove it works,” and the frameworks covered in this series are building the infrastructure for that transition.

Try this today (15 minutes)

Pick one recent “AI-fixed” PR (or one agent-generated change) and run this mini verification loop:

1. Write 3–5 **observable truths** the change must satisfy (user-visible outcomes, not implementation tasks).

2. For each truth, define a **verification method** (test, curl command, query, trace, screenshot, etc.).
3. Run the verifications and record PASS/FAIL in a small `VERIFICATION.md` note.

If any truth fails, don't "fix the symptom." Use a root-cause loop (observe → hypothesize → test → conclude) before touching more code.

Closing: trade trust for proof

AI isn't failing because it's "dumb." It fails because we keep asking it to *finish tasks* instead of *prove outcomes*.

If you take one idea from this series, make it this: **build a verification layer that makes correctness a deliverable.**

If you try goal-backward verification this week, leave a comment with:

- the feature you verified,
- the 3–5 truths you used,
- and what surprised you.

That discussion (what actually fails, and what verification catches) is where the craft advances.

Trade Trust for Proof on Your Next Pull Request

Stop asking AI to finish tasks. Start demanding it proves outcomes.

The 15-Minute Verification Loop:

- ✓ 1. Write 3–5 observable truths (user-visible outcomes).
- ✓ 2. Define a verification method for each (test, curl, query).
- ✓ 3. Record PASS/FAIL in a `VERIFICATION.md` note.



The Golden Rule:

If a truth fails, initiate the Observe → Hypothesize → Test loop before modifying a single line of code.

Related Stories

1. Superpowers: The Psychology Hack That Makes LLMs Obey Engineering Discipline (No More Skipped...) by Rick Hightower

- Date: Apr 14, 2026 (19h ago as of now)
- Link: <https://medium.com/towards-artificial-intelligence/superpowers-the-psychology-hack-that-makes-llms-obey-engineering-discipline-no-more-skipped-0d76dd5f5e9e>

2. What Is GSD? Spec-Driven Development Without the Ceremony by Rick Hightower

- Date: Feb 22, 2025
- Link: <https://medium.com/spillwave-solutions/what-is-gsd-spec-driven-development-without-the-ceremony-570216956a84>

3. From 'Vibe Coding' to Spec-Driven Development: Master GitHub Spec Kit in 2025 by Rick Hightower

- Date: Nov 9, 2025
- Link: <https://medium.com/spillwave-solutions/from-vibe-coding-to-spec-driven-development-master-github-spec-kit-in-2025-f1858a7f44e6>

4. Agentic Coding: GSD vs Spec Kit vs OpenSpec vs Taskmaster AI: Where SDD Tools Diverge by Rick Hightower

- Date: Feb 26, 2026

Open in app ↗

- Date: Mar 6, 2026
- Link: <https://medium.com/ai-in-plain-english/scientific-debugging-with-ai-how-gsds-gsd-debug-command-works-328320afa579>

6. From “Vibe Coding” to Viable Coding: How Superpowers Turn AI Chatbots into Engineering Partners by Rick Hightower

- Date: Mar 14, 2026
- Link: <https://medium.com/towards-artificial-intelligence/from-vibe-coding-to-viable-coding-how-superpowers-turns-ai-chatbots-into-engineering-partners-59c081f02a26>

7. The Great Framework Showdown: Superpowers vs. BMAD vs. SpecKit vs. GSD by Rick Hightower

- Date: Mar 16, 2026
- Link: <https://medium.com/ai-in-plain-english/the-great-framework-showdown-superpowers-vs-bmad-vs-speckit-vs-gsd-360983101c10>

About the Author

Rick Hightower is a Java Champion, former Senior Distinguished Engineer at a fortune 100 focusing on delivering ML / AI insights to front line applications, and practitioner building multi-agent production systems. Follow him on Medium for more hands-on agent engineering content.



Rick Hightower

Medium

Claude Certified Architect Exam

Rick Hightower

Harness Engineering

9 stories

A grid of various diagrams and images related to architecture and engineering, including a network diagram, a flowchart, and a technical drawing.

He created skilz, the [universal agent skill installer](#), supporting 30+ coding agents including Claude Code, Gemini, Copilot, and Cursor, and co-founded the world's largest agentic skill marketplace. Connect with Rick Hightower on [LinkedIn](#) or [Medium](#). Check out [SpillWave](#), your source for AI expertise.

Rick has been actively developing generative AI systems, agents, and agentic workflows for years. He is the author of numerous agentic frameworks and developer tools and brings deep practical expertise to teams looking to adopt AI. He enjoys writing about himself in the 3rd person.

Rick also wrote a [Claude Certified Architect \(CCA\)](#) series of articles that have a lot of useful information on writing agentic AI systems. A lot of ideas captured in the CCA and the exam prep that Rick wrote echoes what you see in this article. If you want to improve your ability to create well-behaved AI agents, studying for the CCA Exam is a good place to start.

CCA Exam Prep on Agentic Development

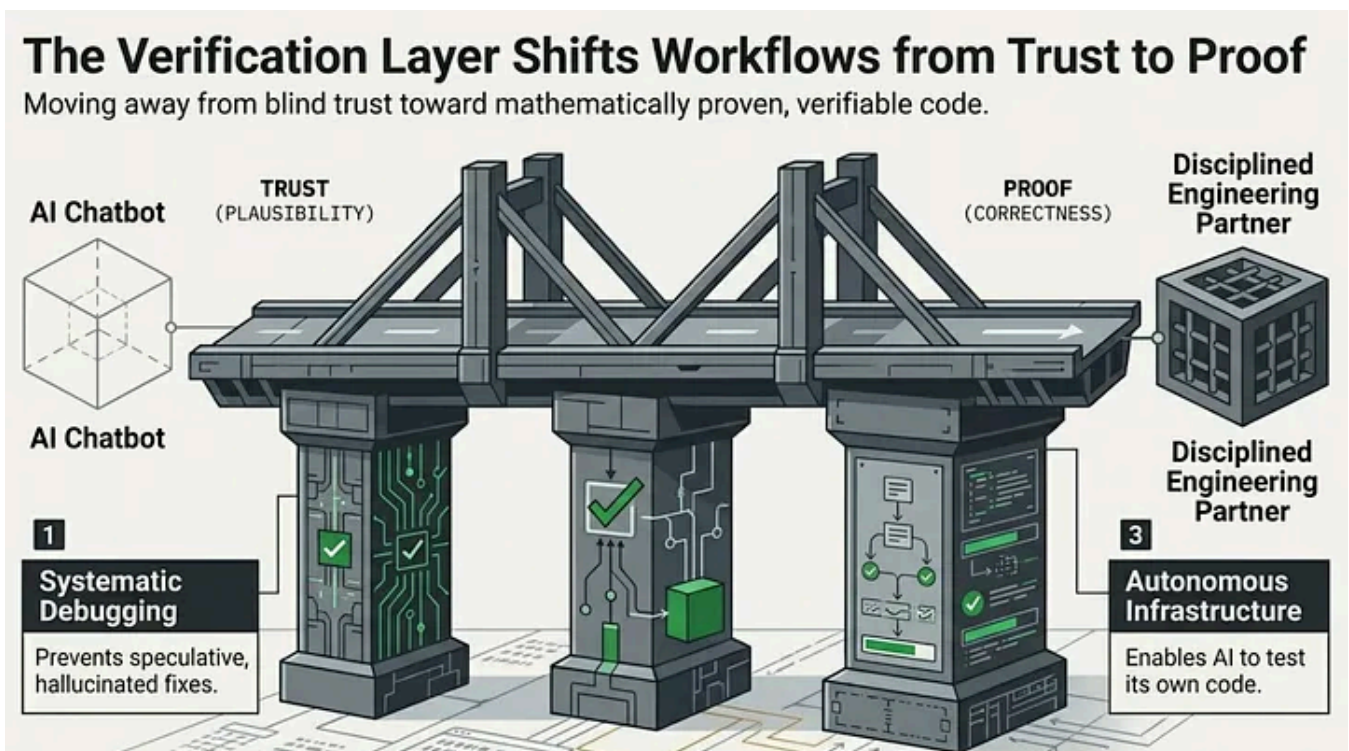
- [Claude Certified Architect: The Complete Guide to Passing the CCA Foundations Exam](#)
- [CCA Exam Prep: Mastering the Code Generation with Claude Code Scenario](#)
- [CCA Exam Prep: Mastering the Multi-Agent Research System Scenario](#)
- [CCA Exam Prep: Structured Data Extraction](#)
- [CCA: Master the Developer Productivity Scenario](#)
- [Claude Certified Architect: Master the CI/CD Scenario](#)
- [CCA Exam Prep: Mastering the Customer Support Resolution Agent Scenario](#)

Rick also wrote a series on harness engineering and how to improve agentic systems using harness engineering for feedback loops and adversarial agents. These articles also go hand in hand with this article.

Harness Engineering Articles

- [The \\$9 Disaster: What Anthropic's Harness Design Paper Teaches Us About Building Autonomous AI](#)
- [Harness Engineering vs Context Engineering: The Model is the CPU, the Harness is the OS](#)

- [LangChain Deep Agents: Harness and Context Engineering: Memory, Skills, and Security](#)
- [Beyond the AI Coding Hangover: How Harness Engineering Prevents the Next Outage](#)
- [LangChain's Harness Engineering: From Top 30 to Top 5 on Terminal Bench 2.0](#)
- [Anthropic's Harness Engineering: Two Agents, One Feature List, Zero Context Overflow](#)
- [OpenAI's Harness Engineering Experiment: Zero Manually-Written Code](#)



Agentic Engineering

AI

AI Agent

Claude Code

Codex Cli



Following

Published in Towards AI

126K followers · Last published just now

We build Enterprise AI. We teach what we learn. Join 100K+ AI practitioners on Towards AI Academy. Free: 6-day Agentic AI Engineering Email Guide: <https://email-course.towardsai.net/>



Following ▾

Written by Rick Hightower

2.8K followers · 79 following

2026 Agent Reliability Playbook – Free Download DM me 'PLAYBOOK' for the full version + personalized 15-minute audit of your current agent setup (no pitch).

No responses yet



★ Christian Broberg

What are your thoughts?

More from Rick Hightower and Towards AI

Mastering the Claude Certified Architect (CCA) Foundations Exam

This certification tests the ability to architect production-ready systems using Claude, focusing on agentic architecture, tool integration, and programmatic reliability rather than just prompt engineering.

COMPETENCY DOMAINS & LOGISTICS

Agentic Architecture (27%)

Focus on multi-agent orchestration and explicit context passing.

MCP: Tools vs. Resources (18%)

Tests distinguishing executable actions (Tools) from read-only context (Resources).

Executable Actions (Tools)

Read-only Context (Resources)

Programmatic Enforcement Over Prompts (20%)

20% of the exam rewards using tool_choice and validation loops instead of prompt instructions.

EXAM LOGISTICS



Format:
60 Multiple-Choice Questions



Time Limit:
120 Minutes



Passing Score:
720 / 1,000

THE 4-WEEK STUDY STRATEGY

Week 4: Simulation & Anti-Patterns

Take the practice exam and memorize "what NOT to do" patterns.

Week 3: MCP & Claude Code

Create an MCP server and configure CLAUDE.md files for project-level standards.

Weeks 1-2: Build the Core

Focus on building validation-retry loops and mastering the tool_use

In Towards AI by Rick Hightower

Claude Certified Architect: The Complete Guide to Passing the CCA Foundations Exam

Everything You Need to Know to Ace the CCA Foundations Exam on Your First Try — Article 1 of 8

★ Mar 24 🖱️ 478 💬 5



If You Understand These 5 AI Terms, You're Ahead of 90% of People.

1. Tokens

The basic units of language data (words or subwords) that an AI model processes.

2. Context Window

The amount of previous text data the AI can actively consider and recall for a single response.

3. Temperature

A control setting for the randomness and creativity in AI-generated output. Higher = more creative, lower = more predictable.

4. Hallucination

When an AI confidently generates incorrect, fabricated, or nonsensical information. A current major challenge.

5. RAG (Retrieval-Augmented Generation)

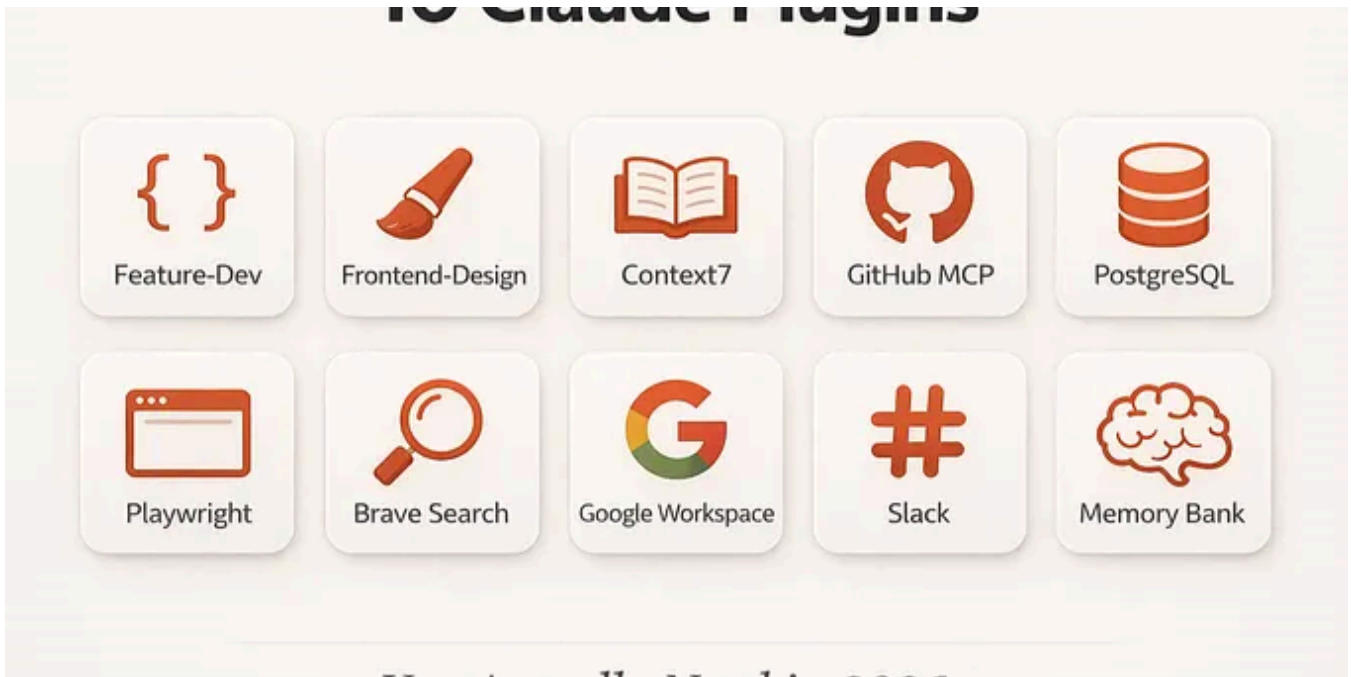
A method that connects an AI to external, trusted data sources before generation to ground its answers and minimize hallucinations.

TOP 10%

In Towards AI by Shreyas Naphad

If You Understand These 5 AI Terms, You're Ahead of 90% of People

Master the core ideas behind AI without getting lost



 In Towards AI by Rohan Mistry

The 10 Claude Plugins You Actually Need in 2026 (And What They Are)

How to transform Claude from a chatbot into an AI that can code, browse, access your data, and automate your entire workflow



 In Towards AI by Rick Hightower

Claude Code Agent Skills 2.0: From Custom Instructions to Programmable Agents

Skills are no longer instructions. They are programs.

 Mar 9  451  9

See all from Rick Hightower

See all from Towards AI

Recommended from Medium

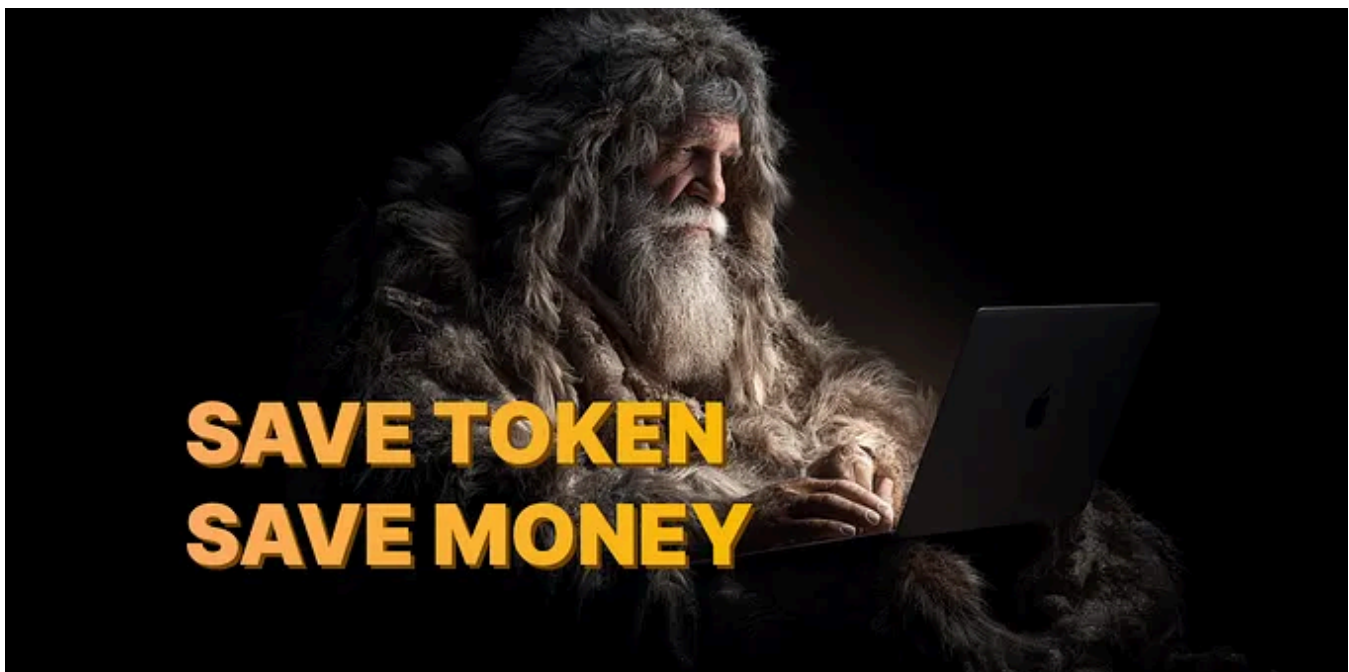
I USED KARPATHY'S LLM WIKI TO BUILD A KNOWLEDGE BASE THAT MAINTAINS ITSELF WITH AI

 Balu Kosuri

I used Karpathy's LLM Wiki to build a knowledge base that maintains itself with AI

Here is a scenario most knowledge workers know too well.

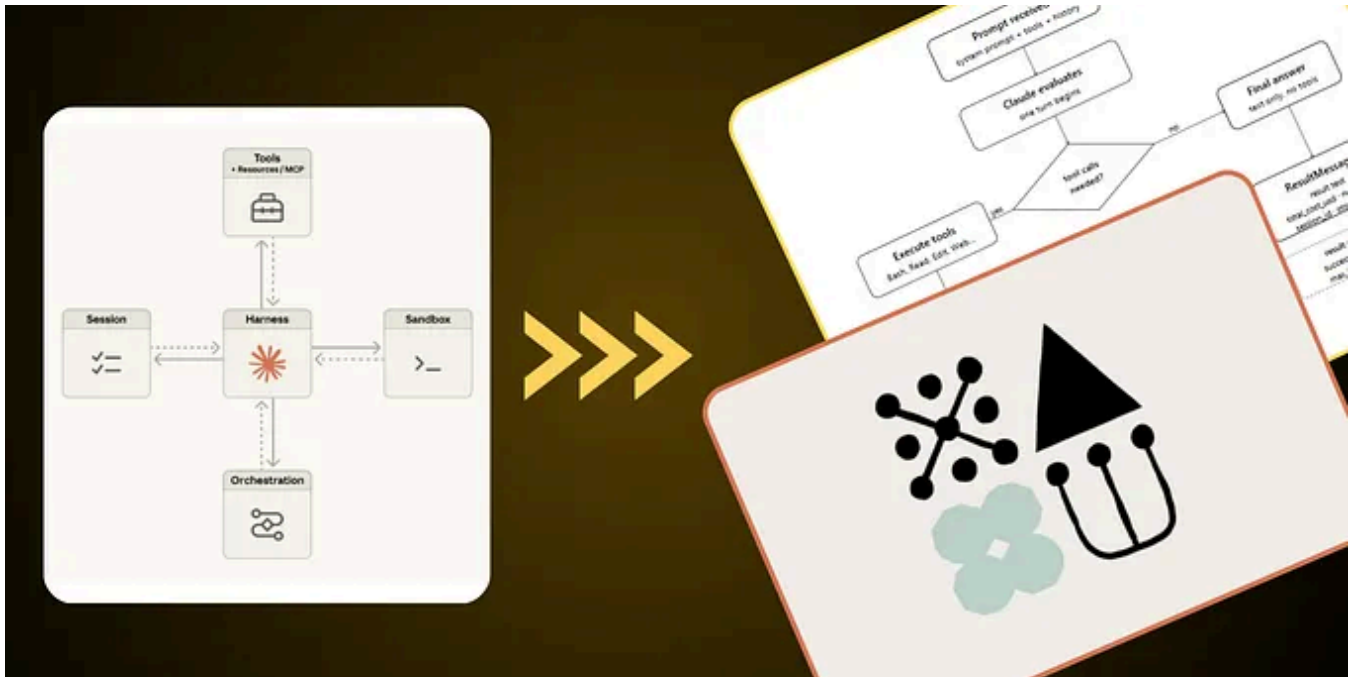
★ Apr 7 🖱 742 💬 12



 In Vibe Coding by Alex Dunlop

I Cut Claude Code's Output Tokens by 75%. Why Did Nobody Tell Me?

Fewer tokens. Somehow more accurate. There's a paper.



🧠 In AI Software Engineer by Joe Njenga

Anthropic Launches Claude Managed Agents (That Make Agentic AI Workflows Real)

Claude Managed Agents stops you from wasting time trying to string scripts together to make your agents work.



🧠 In Towards AI by Rick Hightower

Claude Code 2026: The Daily Operating System Top Developers Actually Use

The exact 5-part model, 10-minute routine, slash commands, context hygiene tricks, end-of-day rituals, and elite workflows power users rely...

★ Apr 7 🖱️ 291



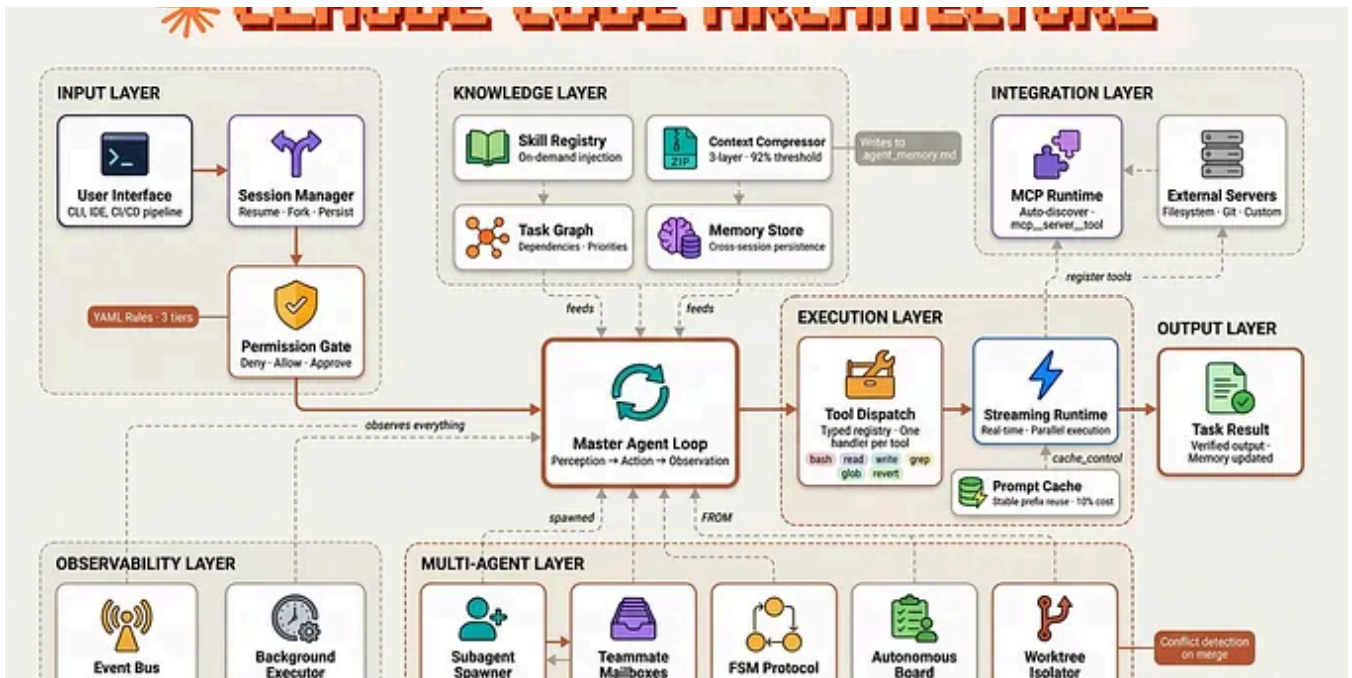
 In AI Advances by Yanli Liu

I Ran Codex and Claude Side by Side. Here's What I Found.

The setup takes 5 minutes. The compliance question takes longer.

★ 6d ago 🖱️ 424 💬 3





 In Level Up Coding by Fareed Khan

Building Claude Code with Harness Engineering

Multi-agents, MCP, skills system, context pipelines and more

★ Apr 6 🤝 1.2K 💬 9



See more recommendations